

CASA BRAVA

ACADEMIA DE COMBATE

MANUAL TÉCNICO DEL BACKEND

API REST de la plataforma Casa Brava

Endpoints · perfiles de usuario y permisos · modelo de datos
pagos en línea con Stripe y dashboard en tiempo real

Versión 2.0 · 23 de septiembre de 2026

Django 6.1 · Django REST Framework 3.18 · Stripe 15.6

Documento de uso interno

Contenido

1. Introducción	4
1.1 Alcance del sistema	4
1.2 Arquitectura	4
1.3 Stack tecnológico	4
1.4 Estructura del proyecto	5
2. Instalación y configuración	6
2.1 Puesta en marcha	6
2.2 Variables de entorno	6
2.3 Convenciones de la API	7
3. Perfiles de usuario y permisos	8
3.1 Alcance del maestro	8
3.2 Cuentas automáticas	8
3.3 Clases de permiso	8
3.4 Matriz de acceso por recurso	9
4. Autenticación	11
5. Modelo de datos	12
5.1 Reglas de integridad relevantes	12
5.2 Diccionario de relaciones	13
6. Referencia de endpoints	18
6.1 Autenticación	18
6.2 Dashboard general	18
6.3 Alumnos	18
6.4 Maestros y cuentas administrativas	19
6.5 Catálogos	20
6.6 Caja: pagos de membresía y ventas	21
6.7 Pagos en línea (Stripe)	21
6.8 Asistencias y check-in	22
6.9 Ficha deportiva, torneos, grados y evaluación MMA	22
6.10 Eventos	23
6.11 Notificaciones y avisos	23
6.12 Ejemplo: check-in por QR	24

- 7. Pagos en línea con Stripe 25**
 - 7.1 Flujo completo 25
 - 7.2 Modelos 25
 - 7.3 Endpoints 26
 - 7.4 Webhook · POST /api/payments/webhook/stripe/ 27
 - 7.5 Dashboard de pagos en tiempo real 27
 - 7.6 Desarrollo local con Stripe CLI 28
 - 7.7 Seguridad 29
- 8. Automatismos y tareas programadas 30**
 - 8.1 Señales 30
 - 8.2 Comandos de administración 30
 - 8.3 Reglas de negocio destacadas 30
- 9. Pruebas y despliegue 31**
 - 9.1 Pruebas automatizadas 31
 - 9.2 Lista de verificación para producción 31

1. Introducción

Este manual describe el backend de la plataforma Casa Brava: una API REST en Django que da servicio a la PWA en Angular (administración, maestros y alumnos), a la tablet de recepción que registra asistencias por código QR y a las tareas programadas del servidor. Está dirigido a quienes desarrollan o mantienen el backend y a quienes consumen la API desde el frontend.

1.1 Alcance del sistema

- **Academia:** alumnos, maestros, horarios (grupos), disciplinas y membresías.
- **Caja:** pagos de membresía (mueven el vencimiento del alumno) y ventas de otros conceptos.
- **Pagos en línea:** cobro de membresías con Stripe Checkout, confirmado por webhook, con dashboard en tiempo real.
- **Operación diaria:** check-in por QR, registro manual y desde la app; recordatorios de pago por correo.
- **Desarrollo deportivo:** ficha técnica, torneos, grados, eventos con inscripción y Evaluación MMA por habilidad.
- **Gamificación:** puntos, rachas de asistencia e insignias con criterios configurables.
- **Comunicación:** notificaciones y avisos a un alumno, a un grupo o a toda la academia.

1.2 Arquitectura

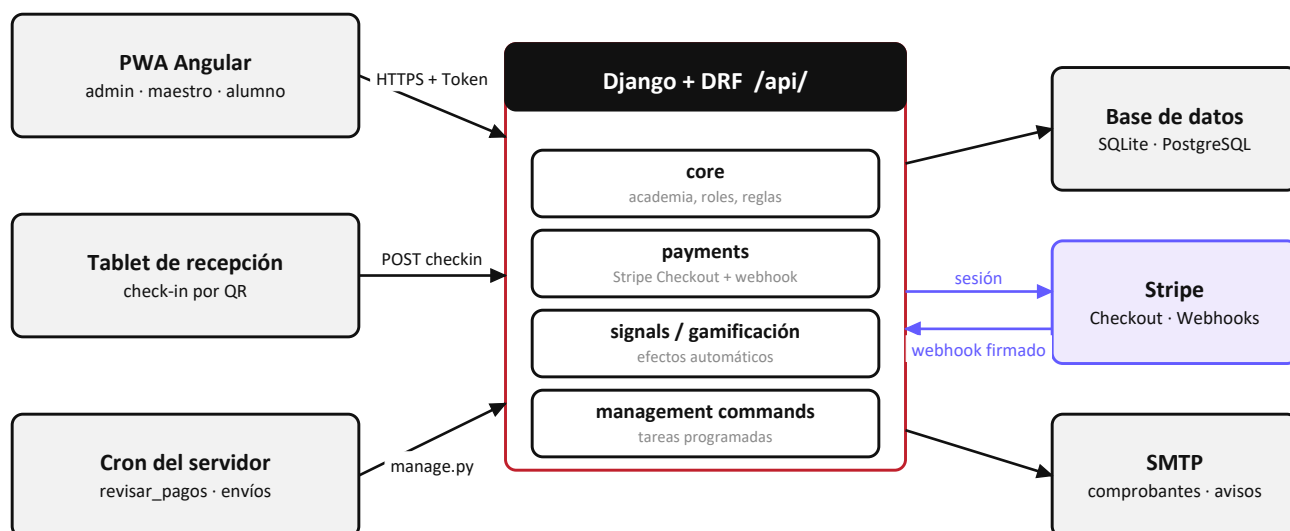


Figura 1. Componentes del sistema y cómo se comunican.

El backend es un único proyecto Django con dos aplicaciones: **core** (toda la lógica de la academia) y **payments** (integración con Stripe). La autenticación es por token de DRF; no se usan cookies de sesión. El backend es la autoridad sobre precios, alcance por rol y estado de los pagos: ocultar algo en Angular nunca cuenta como seguridad.

1.3 Stack tecnológico

Componente	Versión	Uso
Python	3.13	Lenguaje
Django	6.1.1	Framework web, ORM, admin, migraciones
djangorestframework	3.18.1	ViewSets, serializers, token auth, throttling
django-filter	26.1	Filtros por query string
django-cors-headers	4.9.0	CORS para la PWA
python-decouple	3.8	Lectura del .env
qrcode + pillow	8.2 / 12.3.0	Códigos QR de alumnos y maestros; imágenes
xhtml2pdf	0.2.18	Comprobantes de pago en PDF
stripe	15.6.1	Checkout Sessions, webhooks firmados, reembolsos, saldo
PostgreSQL (prod.)	psycopg 3.3.5	Base de datos de producción (SQLite en desarrollo)
gunicorn + whitenoise	26.2.0 / 6.12.0	Servidor WSGI y estáticos en producción

1.4 Estructura del proyecto

```

academia_backend/
├── config/                settings.py (entorno, DRF, CORS, correo, Stripe), urls.py raíz
├── core/                  Aplicación principal
│   ├── models.py          Modelos de la academia (24 tablas)
│   ├── serializers.py      Validaciones y representación de la API
│   ├── views.py           ViewSets y endpoints de negocio
│   ├── auth_views.py       Login, logout, yo, cambiar-password
│   ├── permissions.py      Roles y clases de permiso
│   ├── signals.py         Efectos automáticos (cuentas, puntos, récord, comprobantes)
│   ├── gamificacion.py     Puntos, rachas e insignias
│   ├── evaluacion_mma.py   Cálculo de la Evaluación MMA
│   ├── avisos.py / email_service.py  Notificaciones y correo
│   └── management/commands/ seed, revisar_pagos, enviar_notificaciones_pendientes
├── payments/              Pagos en línea con Stripe
│   ├── models.py          PagoStripe, EventoStripe
│   ├── views.py           checkout, sesion, dashboard, cambios, saldo, reembolsar
│   ├── services/          stripe_service.py (única llamada a Stripe), activacion.py
│   ├── webhooks/stripe.py Receptor firmado e idempotente
│   └── tests/              Checkout, webhooks y dashboard
├── templates/correos/     Plantillas HTML/TXT de correo
└── docs/                  Este manual y su generador

```

2. Instalación y configuración

2.1 Puesta en marcha

```
python -m venv .venv && .venv\Scripts\activate # Windows (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt # o requirements-dev.txt / requirements-prod.txt
copy .env.example .env # y ajustar valores
python manage.py migrate
python manage.py seed # catálogos del cartel y datos demo
python manage.py createsuperuser
python manage.py runserver 0.0.0.0:8000
```

2.2 Variables de entorno

Todas se leen del archivo `.env` con `python-decouple`. Ningún secreto vive en el código ni en el frontend.

Variable	Por defecto	Descripción
SECRET_KEY	clave de desarrollo	Obligatoria y aleatoria en producción.
DEBUG	True	False en producción: activa HSTS, cookies seguras y redirección HTTPS.
ALLOWED_HOSTS	*	Dominios que sirve la API (ej. <code>api.casabrava.mx</code>).
TIME_ZONE	America/Mexico_City	Zona horaria de fechas de pago, asistencias y vencimientos.
DB_ENGINE / DB_*	sqlite	postgresql + DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT.
CORS_ALLOWED_ORIGINS	(vacío)	Orígenes del frontend. En DEBUG se permite cualquiera.
CSRF_TRUSTED_ORIGINS	<code>http://localhost:4200</code>	Origen del frontend para peticiones POST.
FRONTEND_URL	<code>http://localhost:4200</code>	Base de las ligas dentro de los correos.
PASSWORD_INICIAL	rotoplas	Contraseña con la que nacen las cuentas creadas por el sistema.
EMAIL_BACKEND, EMAIL_HOST, EMAIL_PORT, EMAIL_USE_TLS / EMAIL_USE_SSL, EMAIL_HOST_USER, EMAIL_HOST_PASSWORD, DEFAULT_FROM_EMAIL, EMAIL_TIMEOUT	consola · 587 · TLS · 15 s	Envío de comprobantes, avisos y recordatorios. TLS y SSL son excluyentes.
STRIPE_SECRET_KEY	(vacío)	<code>sk_test_...</code> o <code>sk_live_...</code> . Sin ella el checkout responde 503.
STRIPE_WEBHOOK_SECRET	(vacío)	<code>whsec_...</code> del endpoint de webhook. Sin él el webhook responde 503.
STRIPE_CURRENCY	mxn	Moneda de cobro.
STRIPE_SUCCESS_URL / STRIPE_CANCEL_URL	FRONTEND_URL/payment/success cancel	A dónde regresa Stripe al usuario.
THROTTLE_CHECKOUT	10/min	Límite de peticiones al checkout por usuario.
LOG_LEVEL_PAGOS	INFO	Nivel del logger payments (solo ids y estatus).

2.3 Convenciones de la API

- Base: `/api/`. Formato JSON. Fechas `YYYY-MM-DD`, fechas-hora ISO-8601 con zona horaria.
- Autenticación: encabezado `Authorization: Token <token>` en toda petición salvo las públicas.
- Paginación: `{count, next, previous, results}`, 25 elementos por página; `?page=` y `?page_size=` (máx. 500).
- Filtros: `?campo=valor`; rangos con `__gte` / `__lte`; búsqueda con `?search=`; orden con `?ordering=-campo` donde aplique.
- Importes: decimales como texto con dos cifras (`"800.00"`). Moneda MXN.

Códigos de respuesta

Código	Significado en esta API
200 / 201	Correcto / recurso creado.
202	Solicitud aceptada pero se confirma después (reembolso de Stripe).
204	Borrado correcto.
400	Datos inválidos: el cuerpo trae el campo y el motivo.
401	Falta el token o es inválido.
403	El rol no tiene permiso, o el recurso está fuera de su alcance (alumno dado de baja en check-in).
404	No existe, o no es visible para el rol (el alcance se aplica en el queryset).
409	Conflicto: asistencia duplicada de la misma clase el mismo día.
429	Demasiadas peticiones al checkout (throttling).
502	Stripe rechazó o no respondió la operación.
503	Pagos en línea sin configurar (faltan llaves de Stripe).

3. Perfiles de usuario y permisos

Todos los perfiles comparten el modelo **User** de Django. El rol no se guarda en un campo: se deduce de con qué está ligada la cuenta. **GET /api/auth/yo/** y el login devuelven el rol ya resuelto.

Rol	Cómo se determina	Qué puede hacer
ADMINISTRATIVO	<code>User.is_staff = True</code>	Acceso total: catálogos, alumnos, maestros, caja, reportes, avisos a todos, pagos en línea, reembolsos y dashboards.
MAESTRO	Tiene un Maestro ligado (<code>Maestro.usuario</code>)	Trabaja solo con sus alumnos a cargo : los ve, escribe su ficha técnica y Evaluación MMA, los inscribe a eventos, captura resultados y les manda avisos (individual o a sus grupos). Nunca edita datos personales ni pagos.
ALUMNO	Tiene un Alumno ligado (<code>Alumno.usuario</code>)	Lee solo lo suyo; edita sus datos personales en <code>/api/alumnos/yo/</code> ; hace check-in desde la app; se inscribe a eventos; paga su membresía en línea.
SIN_ROL	Cuenta sin ninguna de las anteriores	No ve nada.
Público	Sin token	Login, check-in por QR (tablet) y webhook de Stripe (con firma).

3.1 Alcance del maestro

Lo define el administrativo al editar al maestro: grupos completos (`Maestro.horarios`) más alumnos sueltos (`Maestro.alumnos_asignados`). `Maestro.alumnos_a_cargo()` devuelve la unión de alumnos activos y cada vista la aplica en su queryset, sin confiar en filtros del cliente. Las disciplinas del maestro son solo catálogo (qué imparte) y no dan visibilidad.

3.2 Cuentas automáticas

- Todo alumno y maestro nace con cuenta: usuario `alumno-<id>/maestro-<id>` y contraseña `PASSWORD_INICIAL` (guardada con hash). El alta devuelve `username` y `password_inicial` una sola vez.
- El correo no es único (hermanos pueden compartir el del tutor); el login acepta usuario o correo y prueba cada cuenta con ese correo.
- Cada usuario cambia su contraseña en `POST /api/auth/cambiar-password/`; el administrativo la restablece con `POST /api/<alumnos|maestros|usuarios>/{id}/restablecer-password/`.
- Cambiar o restablecer la contraseña invalida los tokens anteriores.

3.3 Clases de permiso

Clase	Lectura	Escritura	Se usa en
<code>EsAdministrativo</code>	Admin	Admin	usuarios, ventas, dashboard, reportes
<code>EsPersonal</code>	Admin, Maestro	Admin, Maestro	avisos, alumnos por grupo
<code>EsAdministrativoOSoloLectura</code>	Autenticado	Admin	horarios, disciplinas, membresías, insignias, categorías MMA
<code>EsAdministrativoOPersonalSoloLectura</code>	Admin, Maestro	Admin	maestros

Clase	Lectura	Escritura	Se usa en
EsDuenoOAdministrativo	Admin, Alumno dueño	Admin	pagos, asistencias, torneos, grados
PermisoAlumno	Admin, Maestro (alcance), Alumno dueño	Admin	alumnos, notificaciones
EsDuenoOPersonal	Admin, Maestro (alcance), Alumno dueño	Admin, Maestro	experiencias, evaluaciones MMA
EsPersonalOSoloLectura	Autenticado	Admin; Maestro solo sus eventos	eventos
PermisoInscripcionEvento	Admin, Maestro, Alumno dueño	Personal; Alumno POST/DELETE propios	inscripciones a eventos
EsAdministrativoOAlumno	Admin, Alumno dueño	Admin, Alumno (checkout propio)	pagos en línea

3.4 Matriz de acceso por recurso

L = lectura, E = escritura, — = sin acceso. "Propio" significa que el queryset filtra a los registros del alumno autenticado; "alcance", a los alumnos a cargo del maestro.

Recurso	Admin	Maestro	Alumno	Público
Autenticación (login)	E	E	E	E
Dashboard general	L	—	—	—
Alumnos	L / E	L (alcance)	L propio; E en /yo/	—
Morosos, por vencer	L	—	—	—
Ranking	L	L (alcance)	L	—
Maestros	L / E	L	—	—
Cuentas administrativas	L / E	—	—	—
Catálogos (horarios, disciplinas, membresías, insignias, categorías MMA)	L / E	L	L	—
Pagos de membresía (caja)	L / E	—	L propio	—
Ventas	L / E	—	—	—
Pagos en línea: checkout	E (a cualquier alumno)	—	E (propio)	—
Pagos en línea: dashboard, cambios, saldo, reembolso	L / E	—	—	—
Asistencias	L / E	—	L propio	—
Check-in QR	E	E	E	E
Check-in manual / del día	E / L	—	—	—
Mi check-in (app)	—	—	E	—
Experiencia y Evaluación MMA	L / E	L / E (alcance)	L propio	—

Recurso	Admin	Maestro	Alumno	Público
Torneos y grados	L / E	—	L propio	—
Eventos	L / E	L / E (propios)	L	—
Inscripciones a eventos	L / E	L / E (alcance)	L / inscribirse / cancelar propio	—
Notificaciones	L / E	L (de sus avisos)	L propio; marcar leída	—
Avisos	L / E (incl. a todos)	L / E (sus alumnos y grupos)	—	—
Webhook de Stripe	—	—	—	Solo con firma de Stripe

4. Autenticación

La API usa **TokenAuthentication** de DRF. El frontend guarda el token y lo envía en cada petición. No se usa SessionAuthentication porque exige CSRF incluso a peticiones anónimas y rompía el login.

Iniciar sesión

```
POST /api/auth/login/
{"username": "alumno-12", "password": "*****"} // también acepta el correo

200 OK
{
  "token": "9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b",
  "usuario": {
    "id": 31, "username": "alumno-12", "email": "ana@correo.com", "nombre": "Ana Ruiz",
    "rol": "ALUMNO", // ADMINISTRATIVO | MAESTRO | ALUMNO | SIN_ROL
    "alumno_id": 12, "maestro_id": null,
    "alumno": { ...perfil completo del alumno... }
  }
}

401 usuario o contraseña incorrectos · 403 cuenta desactivada
```

Uso del token

```
GET /api/auth/yo/
Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b
```

Endpoint	Cuerpo	Respuesta
POST /api/auth/logout/	—	{detail}; borra el token.
GET /api/auth/yo/	—	Mismo objeto usuario que el login.
POST /api/auth/cambiar-password/	{actual, nueva} (mín. 8 caracteres)	{detail, token} con token nuevo.

5. Modelo de datos

El sistema tiene 25 modelos: 23 en **core** y 2 en **payments**. El alumno es el centro del modelo: casi todo lo operativo cuelga de él. La figura muestra las relaciones principales; las tablas que siguen se generan del propio código e incluyen todas.

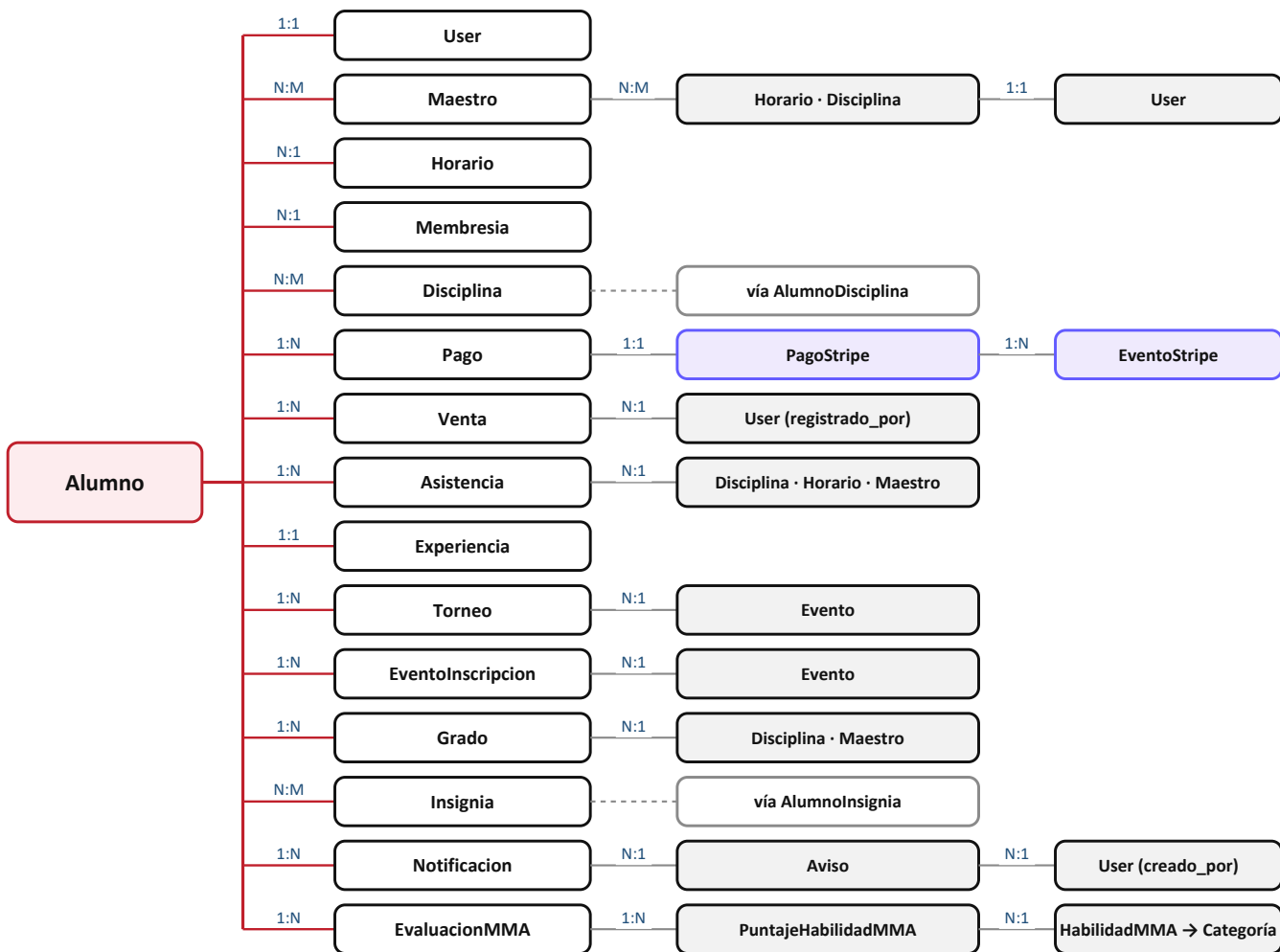


Figura 2. Relaciones principales entre entidades (en rojo, el alumno; en morado, pagos en línea).

5.1 Reglas de integridad relevantes

- **CASCADE** desde Alumno: al borrar un alumno se borran sus pagos, asistencias, experiencia, torneos, grados, notificaciones, evaluaciones e inscripciones. En operación normal se da de baja con **activo=false**.
- **PROTECT** en PagoStripe → Alumno/Membresía: no se puede borrar un alumno o plan con pagos en línea (auditoría financiera).
- **SET_NULL** en catálogos (horario, membresía, disciplina) y en autores (creado_por, registrado_por): el historial sobrevive.
- Unicidades: una asistencia por alumno/clase/día (**asistencia_unica_por_clase**, también validada con NULL), un resultado por alumno y evento, una evaluación MMA por alumno y día, una inscripción por alumno y

evento, un horario por franja.

- Idempotencia de Stripe: `EventoStripe.event_id`, `PagoStripe.stripe_checkout_session_id` y `stripe_payment_intent_id` son únicos.

5.2 Diccionario de relaciones

Horario · Horario · tabla core_horario

Campos: id, creado_en, actualizado_en, hora_inicio, hora_fin, turno, dias, nombre

Restricciones: horario_unico

Disciplina · Disciplina · tabla core_disciplina

Campos: id, creado_en, actualizado_en, nombre, descripcion

Membresia · Membresía · tabla core_membresia

Campos: id, creado_en, actualizado_en, nombre, duracion_dias, precio, descripcion

Maestro · Maestro · tabla core_maestro

Campos: id, creado_en, actualizado_en, nombre, edad, telefono, foto, codigo_qr, qr_imagen, activo

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
usuario	1:1	User	SET_NULL	Sí	maestro
disciplinas	N:M	Disciplina	vía MaestroDisciplina	No	maestros
horarios	N:M	Horario	—	No	maestros
alumnos_asignados	N:M	Alumno	—	No	maestros_asignados

MaestroDisciplina · Disciplina del maestro · tabla core_maestrodisciplina

Campos: id, fecha_inicio, es_titular

Restricciones: maestro_disciplina_unica

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
maestro	N:1	Maestro	CASCADE	No	asignaciones
disciplina	N:1	Disciplina	CASCADE	No	asignaciones

Alumno · Alumno · tabla core_alumno

Campos: id, creado_en, actualizado_en, nombres, apellidos, apodo, edad, peso_actual, telefono, foto, codigo_qr, qr_imagen, fecha_registro, activo, puntos

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
usuario	1:1	User	SET_NULL	Sí	alumno
horario	N:1	Horario	SET_NULL	Sí	alumnos
membresia	N:1	Membresia	SET_NULL	Sí	alumnos
disciplinas	N:M	Disciplina	vía AlumnoDisciplina	No	alumnos
insignias	N:M	Insignia	vía AlumnoInsignia	No	alumnos

AlumnoDisciplina · Disciplina del alumno · tabla core_alumnodisciplina**Campos:** id, fecha_inicio**Restricciones:** alumno_disciplina_unica

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	inscripciones
disciplina	N:1	Disciplina	CASCADE	No	inscripciones

Pago · Pago · tabla core_pago**Campos:** id, creado_en, actualizado_en, monto, metodo, fecha_pago, fecha_vencimiento, duracion, estatus, nota, comprobante_enviado_en

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	pagos
membresia	N:1	Membresia	SET_NULL	Sí	pagos

Venta · Venta · tabla core_venta**Campos:** id, creado_en, actualizado_en, concepto, monto, metodo, fecha, nota

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	SET_NULL	Sí	ventas
registrado_por	N:1	User	SET_NULL	Sí	ventas_registradas

Experiencia · Experiencia · tabla core_experiencia**Campos:** id, creado_en, actualizado_en, bjj_cinturon, numero_torneos, numero_sparrings, peleas_ganadas, peleas_perdidas, peleas_empatadas, metodo_victoria_favorito, peso_competencia, lesion_activa, detalle_lesion, notas_maestro, racha_asistencia, racha_maxima

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	1:1	Alumno	CASCADE	No	experiencia

Asistencia · Asistencia · tabla core_asistencia**Campos:** id, fecha, hora_registro, metodo_registro, puntos_otorgados**Restricciones:** asistencia_unica_por_clase

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	asistencias
disciplina	N:1	Disciplina	SET_NULL	Sí	asistencias
horario	N:1	Horario	SET_NULL	Sí	asistencias
registrada_por	N:1	Maestro	SET_NULL	Sí	asistencias_registradas

Aviso · Aviso · tabla core_aviso

Campos: id, creado_en, actualizado_en, titulo, mensaje, tipo_destinatario, estado, total_destinatarios, total_enviadas, total_fallidas, procesado_en

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	SET_NULL	Sí	avisos
horario	N:1	Horario	SET_NULL	Sí	avisos
creado_por	N:1	User	SET_NULL	Sí	avisos_creados

Notificacion · Notificación · tabla core_notificacion

Campos: id, tipo, titulo, mensaje, fecha_envio, leida, canal, estado_correo, enviado_en, error_correo, referencia_vencimiento

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	notificaciones
aviso	N:1	Aviso	CASCADE	Sí	notificaciones

Torneo · Torneo · tabla core_torneo

Campos: id, creado_en, actualizado_en, nombre_torneo, fecha, resultado, metodo, notas

Restricciones: torneo_resultado_unico_por_evento_alumno

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	torneos
disciplina	N:1	Disciplina	SET_NULL	Sí	torneos
evento	N:1	Evento	SET_NULL	Sí	resultados

Grado · Grado · tabla core_grado

Campos: id, creado_en, actualizado_en, nombre_grado, fecha_obtencion, notas

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	grados
disciplina	N:1	Disciplina	CASCADE	No	grados
otorgado_por	N:1	Maestro	SET_NULL	Sí	grados_otorgados

Insignia · Insignia · tabla core_insignia

Campos: id, creado_en, actualizado_en, nombre, descripcion, icono, criterio, puntos_bonus, activa

AlumnoInsignia · Insignia del alumno · tabla core_alumnoinsignia

Campos: id, fecha_obtencion

Restricciones: alumno_insignia_unica

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	insignias_ganadas
insignia	N:1	Insignia	CASCADE	No	otorgamientos

Evento · Evento · tabla core_evento**Campos:** id, creado_en, actualizado_en, titulo, tipo, fecha, lugar, descripcion

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
disciplina	N:1	Disciplina	SET_NULL	Sí	eventos
creado_por	N:1	User	SET_NULL	Sí	eventos_creados

EventoInscripcion · Inscripción a evento · tabla core_eventoinscripcion**Campos:** id, fecha_inscripcion, asistio**Restricciones:** evento_alumno_unico

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
evento	N:1	Evento	CASCADE	No	inscripciones
alumno	N:1	Alumno	CASCADE	No	inscripciones_eventos

CategoriaMMA · Categoría MMA · tabla core_categoriamma**Campos:** id, clave, nombre, puntos_maximos, orden, activa**HabilidadMMA** · Habilidad MMA · tabla core_habilidadmma**Campos:** id, nombre, orden, activa**Restricciones:** habilidad_mma_unica_por_categoria

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
categoria	N:1	CategoriaMMA	CASCADE	No	habilidades

EvaluacionMMA · Evaluación MMA · tabla core_evaluacionmma**Campos:** id, creado_en, actualizado_en, fecha, estado, notas**Restricciones:** evaluacion_mma_unica_por_dia

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	CASCADE	No	evaluaciones_mma
evaluador	N:1	Maestro	SET_NULL	Sí	evaluaciones_mma
creado_por	N:1	User	SET_NULL	Sí	evaluaciones_mma_creadas

PuntajeHabilidadMMA · Puntaje de habilidad · tabla core_puntajehabilidadmma**Campos:** id, puntaje**Restricciones:** puntaje_unico_por_habilidad

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
evaluacion	N:1	EvaluacionMMA	CASCADE	No	puntajes
habilidad	N:1	HabilidadMMA	PROTECT	No	puntajes

PagoStripe · Pago en línea · tabla payments_pagostripe

Campos: id, creado_en, actualizado_en, monto, moneda, estatus, referencia, stripe_checkout_session_id, stripe_payment_intent_id, checkout_url, pagado_en, detalle_error

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
alumno	N:1	Alumno	PROTECT	No	pagos_stripe
membresia	N:1	Membresia	PROTECT	No	pagos_stripe
solicitado_por	N:1	User	SET_NULL	Sí	pagos_stripe_solicitados
pago	1:1	Pago	SET_NULL	Sí	stripe

EventoStripe · Evento de Stripe · tabla payments_eventostripe

Campos: id, event_id, tipo, procesado_en

Campo	Tipo	Apunta a	Al borrar destino	Nulo	Relación inversa
pago_stripe	N:1	PagoStripe	SET_NULL	Sí	eventos

6. Referencia de endpoints

La API expone 81 rutas. La tabla de cada módulo se genera del router de Django, por lo que métodos y permisos corresponden exactamente al código. Las rutas de colección aceptan GET (lista paginada) y POST (alta); las de detalle {pk}, GET, PUT, PATCH y DELETE, salvo que se indique otra cosa.

6.1 Autenticación

Método	Ruta	Roles	Descripción
POST	/api/auth/login/	Público (sin token)	Inicia sesión con usuario o correo y contraseña. Devuelve token y perfil con rol.
POST	/api/auth/logout/	Cualquier usuario autenticado	Invalida el token actual.
GET	/api/auth/yo/	Cualquier usuario autenticado	Perfil y rol del usuario autenticado (rehidrata la sesión de la PWA).
POST	/api/auth/cambiar-password/	Cualquier usuario autenticado	Cambia la contraseña propia y emite un token nuevo.
POST	/api/auth/token/	Público (sin token)	Alternativa estándar de DRF: devuelve solo el token.

6.2 Dashboard general

Método	Ruta	Roles	Descripción
GET	/api/dashboard/	Admin	Métricas generales: alumnos, asistencias, ingresos, desglose por disciplina/horario, top 5.

6.3 Alumnos

Método	Ruta	Roles	Descripción
GET POST	/api/alumnos/	Lectura: Admin · Maestro (sus alumnos) · Alumno (lo suyo) Escritura: Admin	Lista (con alcance por rol) y alta de alumnos. El alta crea cuenta y QR.
GET	/api/alumnos/morosos/	Admin	Alumnos activos con pago vencido o sin pagos.
GET	/api/alumnos/por-vencer/	Admin	Alumnos cuyo pago vence en los próximos ?dias=N.
GET	/api/alumnos/ranking/	Cualquier usuario autenticado	Tabla de posiciones por puntos (?limite=20), con el alcance del listado.
GET PATCH	/api/alumnos/yo/	Cualquier usuario autenticado	Perfil del alumno autenticado; PATCH edita solo datos personales y correo.
GET PUT PATCH DELETE	/api/alumnos/{pk}/	Lectura: Admin · Maestro (sus alumnos) · Alumno (lo suyo) Escritura: Admin	Detalle, edición y baja de un alumno.

Método	Ruta	Roles	Descripción
GET	/api/alumnos/{pk}/asistencias/	Admin · Maestro (sus alumnos) · Alumno (lo suyo)	Historial de asistencias del alumno.
GET	/api/alumnos/{pk}/evaluacion-historial/	Admin · Maestro (sus alumnos) · Alumno (lo suyo)	Serie cronológica de evaluaciones MMA, para graficar.
GET	/api/alumnos/{pk}/evaluacion-resumen/	Admin · Maestro (sus alumnos) · Alumno (lo suyo)	Última evaluación MMA finalizada y variación contra la anterior.
POST	/api/alumnos/{pk}/evaluar-insignias/	Admin	Reevalúa y otorga las insignias que el alumno ya cumple.
GET	/api/alumnos/{pk}/insignias/	Admin · Maestro (sus alumnos) · Alumno (lo suyo)	Insignias ganadas por el alumno.
GET	/api/alumnos/{pk}/pagos/	Admin · Alumno (lo suyo)	Pagos de membresía del alumno.
GET	/api/alumnos/{pk}/perfil/	Admin · Maestro (sus alumnos) · Alumno (lo suyo)	Tarjeta completa de peleador: estadísticas, récord y estado de pago.
GET	/api/alumnos/{pk}/qr/	Admin · Maestro (sus alumnos) · Alumno (lo suyo)	Código QR del alumno y URL de su imagen.
POST	/api/alumnos/{pk}/restablecer-password/	Admin	Restablece la contraseña a PASSWORD_INICIAL.

/api/alumnos/ — **Filtros:** activo, horario, membresia, disciplinas · **?search=** nombres, apellidos, apodo, telefono, codigo_qr · **?ordering=** puntos, apellidos, fecha_registro

6.4 Maestros y cuentas administrativas

Método	Ruta	Roles	Descripción
GET POST	/api/maestros/	Lectura: Admin, Maestro Escritura: Admin	Catálogo de maestros; el alta crea su cuenta.
GET PUT PATCH DELETE	/api/maestros/{pk}/	Lectura: Admin, Maestro Escritura: Admin	Detalle y edición de un maestro (disciplinas, grupos y alumnos a cargo).
POST	/api/maestros/{pk}/restablecer-password/	Admin	Restablece la contraseña del maestro.
GET POST	/api/usuarios/	Admin	Cuentas administrativas (is_staff). Sin PUT ni DELETE: se desactivan con PATCH.
GET PATCH	/api/usuarios/{pk}/	Admin	Detalle y edición parcial de una cuenta administrativa.
POST	/api/usuarios/{pk}/restablecer-password/	Admin	Restablece la contraseña de una cuenta administrativa.

/api/maestros/ — **Filtros:** activo, disciplinas, horarios · **?search=** nombre

/api/usuarios/ — **Filtros:** is_active · **?search=** username, first_name, last_name, email · **?ordering=** username, date_joined, last_login

6.5 Catálogos

Método	Ruta	Roles	Descripción
GET POST	/api/horarios/	Lectura: Cualquier autenticado Escritura: Admin	Catálogo de horarios (grupos) del cartel.
GET PUT PATCH DELETE	/api/horarios/{pk}/	Lectura: Cualquier autenticado Escritura: Admin	Detalle y edición de un horario.
GET	/api/horarios/{pk}/alumnos/	Admin, Maestro (con alcance)	Alumnos activos del grupo (el maestro, solo los que tiene a cargo).
GET POST	/api/disciplinas/	Lectura: Cualquier autenticado Escritura: Admin	Catálogo de disciplinas.
GET PUT PATCH DELETE	/api/disciplinas/{pk}/	Lectura: Cualquier autenticado Escritura: Admin	Detalle y edición de una disciplina.
GET POST	/api/membresias/	Lectura: Cualquier autenticado Escritura: Admin	Catálogo de planes con precio y duración (fuente del precio de Stripe).
GET PUT PATCH DELETE	/api/membresias/{pk}/	Lectura: Cualquier autenticado Escritura: Admin	Detalle y edición de un plan.
GET POST	/api/insignias/	Lectura: Cualquier autenticado Escritura: Admin	Catálogo de insignias con su criterio JSON.
GET PUT PATCH DELETE	/api/insignias/{pk}/	Lectura: Cualquier autenticado Escritura: Admin	Detalle y edición de una insignia.
POST	/api/insignias/{pk}/otorgar/	Admin	Otorga una insignia manualmente: {"alumno_id": 3}.
GET POST	/api/categorias-mma/	Lectura: Cualquier autenticado Escritura: Admin	Catálogo de categorías y habilidades de la Evaluación MMA.
GET PUT PATCH DELETE	/api/categorias-mma/{pk}/	Lectura: Cualquier autenticado Escritura: Admin	Detalle y edición de una categoría.

/api/horarios/ — Filtros: turno · ?ordering= hora_inicio, turno

/api/disciplinas/ — ?search= nombre

/api/membresias/ — ?ordering= precio, duracion_dias

/api/insignias/ — Filtros: activa · ?search= nombre

/api/categorias-mma/ — ?search= nombre

6.6 Caja: pagos de membresía y ventas

Método	Ruta	Roles	Descripción
GET POST	/api/pagos/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Pagos de membresía (caja). Mueven vencimiento y semáforo del alumno.
GET	/api/pagos/resumen/	Admin	Corte de caja del mes por método.
GET PUT PATCH DELETE	/api/pagos/{pk}/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Detalle y edición de un pago.
GET	/api/pagos/{pk}/comprobante/	Admin · Alumno (lo suyo)	Recibo en HTML; ?formato=pdf para PDF; ?enviar=1 lo reenvía por correo.
GET POST	/api/ventas/	Admin	Otros ingresos (inscripción, equipo...). No tocan membresías.
GET	/api/ventas/resumen/	Admin	Corte de ventas del mes por método y concepto.
GET PUT PATCH DELETE	/api/ventas/{pk}/	Admin	Detalle y edición de una venta.
/api/pagos/ — Filtros: alumno, estatus, metodo, fecha_pago, fecha_pago_gte, fecha_pago_lte · ?search= alumno__nombres, alumno__apellidos · ?ordering= fecha_pago, monto, fecha_vencimiento /api/ventas/ — Filtros: alumno, metodo, fecha, fecha_gte, fecha_lte · ?search= concepto, alumno__nombres, alumno__apellidos, alumno__apodo · ?ordering= fecha, monto, concepto			

6.7 Pagos en línea (Stripe)

Método	Ruta	Roles	Descripción
POST	/api/payments/webhook/stripe/	Stripe (firma HMAC)	Receptor de eventos de Stripe. Público, pero exige firma Stripe-Signature.
GET	/api/payments/	Admin · Alumno (lo suyo)	Pagos en línea (Stripe). Admin: todos; alumno: los suyos.
GET	/api/payments/cambios/	Admin	Polling en tiempo real: pagos modificados desde ?desde=<cursor>.
POST	/api/payments/checkout/	Admin · Alumno (lo suyo)	Inicia el pago: crea la Checkout Session y devuelve checkout_url.
GET	/api/payments/dashboard/	Admin	Tablero de pagos en línea: resumen, serie diaria, recientes, salud del webhook y cursor.
GET	/api/payments/saldo/	Admin	Saldo disponible y pendiente consultado en vivo a Stripe.
GET	/api/payments/sesion/	Admin · Alumno (lo suyo)	Estado de un pago por ?session_id= (pantalla de éxito).
GET	/api/payments/{pk}/	Admin · Alumno (lo suyo)	Detalle de un pago en línea.
POST	/api/payments/{pk}/reembolsar/	Admin	Solicita a Stripe el reembolso total (se confirma por webhook).
/api/payments/ — Filtros: alumno, estatus, membresia, creado_en_gte, creado_en_lte			

6.8 Asistencias y check-in

Método	Ruta	Roles	Descripción
GET POST	/api/asistencias/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Historial de asistencias.
POST	/api/asistencias/checkin/	Público (sin token)	Check-in por QR desde la tablet de recepción (sin token).
GET	/api/asistencias/hoy/	Admin	Asistencias del día.
POST	/api/asistencias/manual/	Admin	Registro manual por recepción (alumno_id, fecha opcional no futura).
POST	/api/asistencias/mi-checkin/	Cualquier usuario autenticado	El alumno registra su propia asistencia de hoy desde la app.
GET PUT PATCH DELETE	/api/asistencias/{pk}/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Detalle, corrección o borrado (devuelve los puntos).

/api/asistencias/ — **Filtros:** alumno, disciplina, horario, fecha, metodo_registro · **?ordering=** fecha, hora_registro

6.9 Ficha deportiva, torneos, grados y evaluación MMA

Método	Ruta	Roles	Descripción
GET POST	/api/experiencias/	Lectura: Admin · Maestro (con alcance) · Alumno (lo suyo) Escritura: Admin, Maestro (con alcance)	Ficha técnica del peleador (cinturón, récord, lesiones).
GET PUT PATCH DELETE	/api/experiencias/{pk}/	Lectura: Admin · Maestro (con alcance) · Alumno (lo suyo) Escritura: Admin, Maestro (con alcance)	Detalle y edición de la ficha técnica.
POST	/api/experiencias/{pk}/sincronizar-record/	Admin, Maestro (con alcance)	Recalcula el récord G-P-E desde los torneos.
GET POST	/api/torneos/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Resultados de peleas; recalculan el récord del alumno.
GET PUT PATCH DELETE	/api/torneos/{pk}/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Detalle y edición de un resultado.
GET POST	/api/grados/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Grados/cinturones otorgados.
GET PUT PATCH DELETE	/api/grados/{pk}/	Lectura: Admin · Alumno (lo suyo) Escritura: Admin	Detalle y edición de un grado.

Método	Ruta	Roles	Descripción
GET POST	/api/evaluaciones-mma/	Lectura: Admin · Maestro (con alcance) · Alumno (lo suyo) Escritura: Admin, Maestro (con alcance)	Evaluaciones periódicas por habilidad (1-5), con historial.
GET PUT PATCH DELETE	/api/evaluaciones-mma/{pk}/	Lectura: Admin · Maestro (con alcance) · Alumno (lo suyo) Escritura: Admin, Maestro (con alcance)	Detalle y edición de una evaluación (reemplaza la lista de puntajes).
/api/experiencias/ — Filtros: alumno, bjj_cinturon, lesion_activa · ?ordering= racha_asistencia, peleas_ganadas			
/api/torneos/ — Filtros: alumno, disciplina, resultado, evento · ?search= nombre_torneo · ?ordering= fecha			
/api/grados/ — Filtros: alumno, disciplina, otorgado_por · ?ordering= fecha_obtencion			
/api/evaluaciones-mma/ — Filtros: alumno, estado, evaluador · ?ordering= fecha, creado_en			

6.10 Eventos

Método	Ruta	Roles	Descripción
GET POST	/api/eventos/	Lectura: Cualquier autenticado Escritura: Admin · Maestro (sus propios eventos)	Cartelera de torneos, seminarios y exámenes.
GET PUT PATCH DELETE	/api/eventos/{pk}/	Lectura: Cualquier autenticado Escritura: Admin · Maestro (sus propios eventos)	Detalle y edición (el maestro, solo sus eventos).
GET POST	/api/inscripciones-evento/	Lectura: Admin · Maestro (con alcance) · Alumno (lo suyo) Escritura: Admin, Maestro · Alumno: POST/DELETE propios	Inscripciones a eventos. El alumno se inscribe a sí mismo.
GET PUT PATCH DELETE	/api/inscripciones-evento/{pk}/	Lectura: Admin · Maestro (con alcance) · Alumno (lo suyo) Escritura: Admin, Maestro · Alumno: POST/DELETE propios	Detalle; PATCH asistio (personal); DELETE cancela.
POST DELETE	/api/inscripciones-evento/{pk}/ resultado/	Admin, Maestro · Alumno: POST/DELETE propios	Captura (POST) o quita (DELETE) el resultado del alumno en el evento.
/api/eventos/ — Filtros: tipo, disciplina · ?search= titulo, lugar · ?ordering= fecha, creado_en			
/api/inscripciones-evento/ — Filtros: evento, alumno, asistio			

6.11 Notificaciones y avisos

Método	Ruta	Roles	Descripción
GET POST	/api/notificaciones/	Lectura: Admin · Maestro (sus alumnos) · Alumno (lo suyo) Escritura: Admin	Bandeja de notificaciones (el maestro solo ve las de sus avisos).
GET PUT PATCH DELETE	/api/notificaciones/{pk}/	Lectura: Admin · Maestro (sus alumnos) · Alumno (lo suyo) Escritura: Admin	Detalle de una notificación.

Método	Ruta	Roles	Descripción
POST	/api/notificaciones/{pk}/marcar-leida/	Cualquier usuario autenticado	Marca la notificación como leída (solo su dueño).
GET POST	/api/avisos/	Admin, Maestro (con alcance)	Campañas de aviso: individual, grupo o todos (todos, solo Admin).
GET PUT PATCH DELETE	/api/avisos/{pk}/	Admin, Maestro (con alcance)	Detalle y edición mientras siga PENDIENTE.
POST	/api/avisos/{pk}/cancelar/	Admin, Maestro (con alcance)	Cancela los envíos pendientes del aviso.

/api/notificaciones/ — **Filtros:** alumno, tipo, leida, canal, aviso

/api/avisos/ — **Filtros:** tipo_destinatario, estado, horario, alumno · ?search= titulo, mensaje · ?ordering= creado_en, estado

6.12 Ejemplo: check-in por QR

```
POST /api/asistencias/checkin/ (público: tablet de recepción)
{"codigo_qr": "ALU-364F1A7C0D6E", "disciplina_id": 1, "horario_id": 2}

201 Created
{
  "detail": "Asistencia registrada para Ana Ruiz.",
  "alumno": {"id": 13, "nombre": "Ana Ruiz", "puntos": 30, "al_corriente": false, "dias_para_vencer": null},
  "racha": 1, "puntos_otorgados": 10, "insignias_desbloqueadas": ["Primer paso"], "asistencia_id": 150
}
400 QR desconocido · 403 alumno dado de baja · 409 ya registró esa clase hoy
```

7. Pagos en línea con Stripe

Los alumnos pagan su membresía con **Stripe Checkout**: la página de pago la hospeda Stripe, así que el sistema nunca ve ni guarda datos de tarjeta. El frontend solo dice *qué* se compra; el backend decide el precio, crea la sesión con la llave secreta y únicamente el **webhook firmado** confirma el pago y activa la membresía.

7.1 Flujo completo

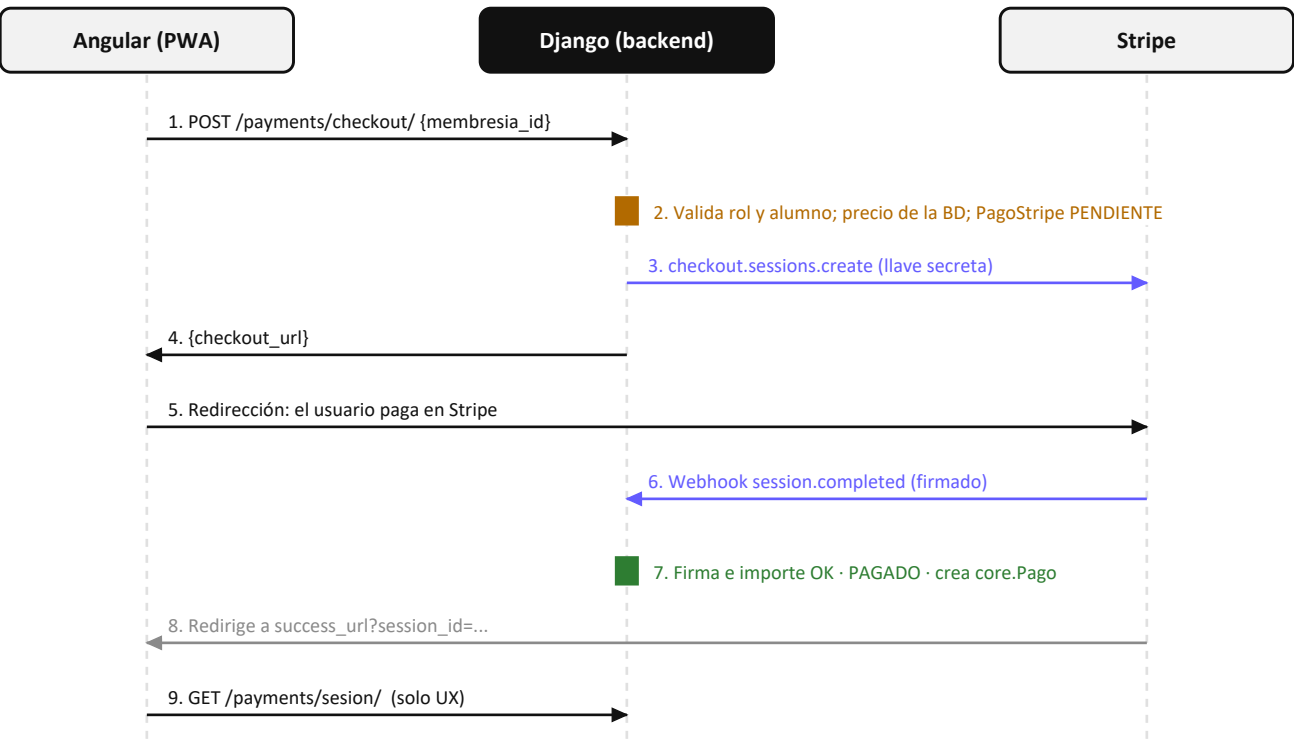


Figura 3. Secuencia de un pago. La success_url es solo experiencia de usuario: nada se activa por visitarla.

7.2 Modelos

Modelo	Propósito	Campos clave
PagoStripe	Un intento de cobro en línea. Nace PENDIENTE; el webhook lo mueve de estado. Al pagarse crea el core.Pago que cubre los días.	alumno, membresia, solicitado_por, monto, moneda, estatus, referencia (UUID), stripe_checkout_session_id, stripe_payment_intent_id, checkout_url, pago (1:1 core.Pago), pagado_en, detalle_error
EventoStripe	Registro de cada evento de webhook ya procesado. Su event_id único garantiza que un reenvío no se aplique dos veces.	event_id, tipo, pago_stripe, procesado_en

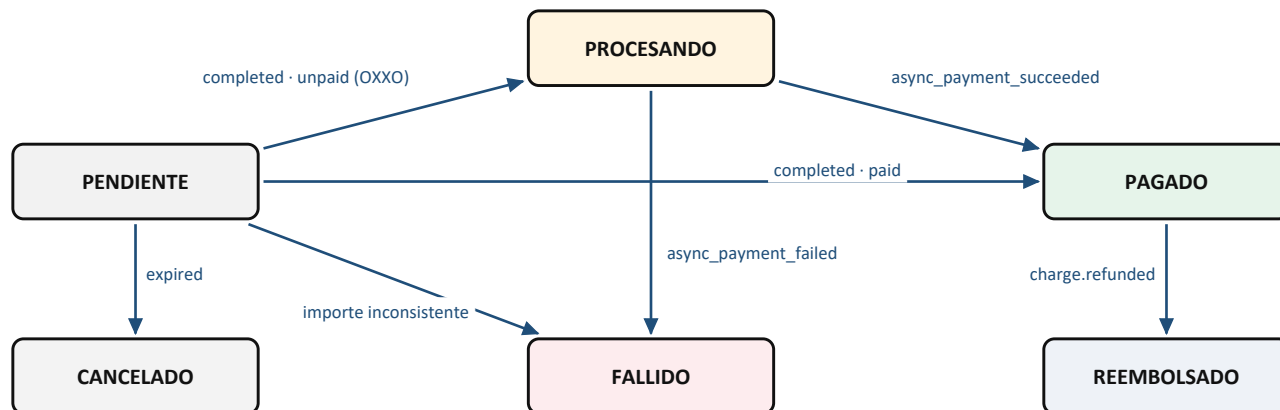


Figura 4. Estados de PagoStripe. Solo el webhook firmado cambia el estado.

7.3 Endpoints

Iniciar pago · POST /api/payments/checkout/

Alumno: paga su propia membresía (sin membresía_id usa la que tiene asignada). Administrativo: genera la liga para cualquier alumno y la puede compartir. Si ya existe una liga PENDIENTE del mismo alumno, plan y precio de menos de 23 horas, se devuelve esa misma (200) en lugar de crear otra. Limitado a 10 peticiones por minuto por usuario.

```

POST /api/payments/checkout/
Authorization: Token <token del alumno>
{"membresia_id": 3} // admin: {"alumno_id": 12, "membresia_id": 3}

201 Created
{
  "id": 41,
  "referencia": "5b1f0a8e-2c1d-4b8e-9d7a-0f3c2e9b6a11",
  "checkout_url": "https://checkout.stripe.com/c/pay/cs_test_a1B2...",
  "monto": "800.00",
  "moneda": "MXN"
}
400 sin membresía / alumno de baja / plan sin precio · 403 pagar por otro alumno o rol maestro
429 demasiadas peticiones · 502 Stripe rechazó la sesión · 503 Stripe sin configurar
  
```

Cualquier **monto** o **amount** que envíe el frontend se ignora: el precio sale de **Membresia.precio**.

Pantalla de éxito · GET /api/payments/sesion/?session_id=cs_...

Devuelve el PagoStripe de esa sesión (solo el dueño o un administrativo). El frontend lo consulta al volver de Stripe y, si sigue PENDIENTE o PROCESANDO, reintenta cada pocos segundos: el webhook suele llegar en 1–5 s.

Listado · GET /api/payments/

Filtros **alumno**, **estatus**, **membresia**, **creado_en_gte**, **creado_en_lte**. Admin ve todos; el alumno, los suyos.

Reembolso · POST /api/payments/{id}/reembolsar/

Solo administrativo y solo pagos PAGADO. Pide a Stripe el reembolso total y responde 202; el estado cambia a REEMBOLSADO cuando llega charge.refunded. Nunca se simula un reembolso editando la base de datos.

Saldo · GET /api/payments/saldo/

```
{"disponible": [{"moneda": "MXN", "monto": "12500.00"}], "pendiente": [{"moneda": "MXN", "monto": "1600.00"}]}
```

7.4 Webhook · POST /api/payments/webhook/stripe/

Es público porque Stripe necesita alcanzarlo, pero se verifica la firma HMAC del encabezado **Stripe-Signature** sobre el cuerpo crudo antes de leer nada (tolerancia de 5 minutos contra repetición). Firma inválida: 400. Todo el procesamiento es una transacción: si algo falla se responde 500, el evento no queda registrado y Stripe lo reintenta.

Evento	Efecto
checkout.session.completed	payment_status = paid: verifica importe y moneda, marca PAGADO y activa la membresía. unpaid (OXXO, transferencia): PROCESANDO.
checkout.session.async_payment_succeeded	Pago diferido confirmado: PAGADO y activación.
checkout.session.async_payment_failed	Pago diferido no completado: FALLIDO.
checkout.session.expired	La sesión venció sin pago: CANCELADO.
charge.refunded	Reembolso total: REEMBOLSADO y se borra el core.Pago (deja de cubrir días). Parcial: se registra y se conserva la membresía.
Cualquier otro	Se registra en EventoStripe y se responde 200 sin efecto.

Activación de la membresía

- Se crea un **core.Pago** con método TARJETA por el monto cobrado; su señal manda el comprobante por correo.
- Si el alumno sigue al corriente, los días se suman a partir de su vencimiento actual (renovar antes no le quita días); si ya venció, cuentan desde hoy.
- Si compró un plan distinto al asignado, se actualiza **Alumno.membresia**.
- Si el importe o la moneda recibidos no coinciden con lo esperado, no se activa nada y el pago queda FALLIDO con el detalle.
- La sesión del evento debe coincidir con la guardada en el PagoStripe; si no, el evento se ignora.

7.5 Dashboard de pagos en tiempo real

El webhook actualiza la base de datos en cuanto Stripe confirma, así que el tablero refleja cada cobro en segundos sin consultar a Stripe en cada carga. El frontend usa dos llamadas, ambas solo para administrativos:

- **GET /api/payments/dashboard/**: foto completa del tablero y un **cursor**.
- **GET /api/payments/cambios/?desde=<cursor>**: los pagos creados o modificados después del cursor y el cursor siguiente. Se consulta cada 3–5 s y se aplica un upsert por id; cuando **hay_cambios** es true se recarga el dashboard para refrescar los totales.

```
GET /api/payments/dashboard/
{
  "cursor": "2026-09-23T17:04:55.120000+00:00",
  "stripe": {"configurado": true, "webhook_configurado": true, "modo": "test", "moneda": "MXN"},
  "resumen": {
    "periodo": "2026-09",
    "cobrado_hoy": {"total": "1600.00", "cantidad": 2},
    "cobrado_mes": {"total": "24000.00", "cantidad": 28},
    "ticket_promedio_mes": "857.14",
    "reembolsado_mes": {"total": "800.00", "cantidad": 1},
    "pendientes": {"total": "2400.00", "cantidad": 3},
    "por_estatus_mes": {"PENDIENTE": 2, "PROCESANDO": 1, "PAGADO": 28,
      "FALLIDO": 1, "CANCELADO": 4, "REEMBOLSADO": 1},
    "tasa_conversion_mes": 75.7
  },
  "serie_diaria": [{"dia": "2026-09-22", "total": "3200.00", "cantidad": 4}, ...], // 30 días
  "por_membresia_mes": [{"membresia": 1, "membresia_nombre": "Mensualidad Básica",
    "total": "16000.00", "cantidad": 20}],
  "recientes": [ ...10 PagoStripe más recientes... ],
  "webhook": {"ultimo_evento": {"tipo": "checkout.session.completed", "procesado_en": "..."},
    "eventos_24h": 9}
}

GET /api/payments/cambios/?desde=2026-09-23T17%3A04%3A55.120000%2B00%3A00
{"cursor": "2026-09-23T17:05:00.410000+00:00", "hay_cambios": true, "pagos": [ {...PagoStripe...} ]}
```

Nota. El cursor lleva un solape de 5 segundos para no perder filas cuya transacción se confirmó justo después de la consulta anterior; por eso un pago puede llegar dos veces y el cliente debe hacer upsert por id. El signo + de la zona horaria debe ir codificado en la URL (%2B); en Angular, `HttpParams` lo hace solo.

Consumo desde Angular (referencia)

```
// payments-dashboard.service.ts
dashboard$ = this.http.get<Dashboard>('/api/payments/dashboard/');

enVivo(cursorInicial: string): Observable<CambiosPagos> {
  let cursor = cursorInicial;
  return timer(0, 4000).pipe(
    switchMap(() => this.http.get<CambiosPagos>('/api/payments/cambios/',
      { params: new HttpParams().set('desde', cursor) })),
    tap(r => cursor = r.cursor),
    filter(r => r.hay_cambios),
  );
}
// En el componente: upsert de r.pagos por id en la tabla y recargar dashboard$ para los totales.
```

La salud del webhook (último evento y eventos en 24 h) permite mostrar una alerta si Stripe dejó de notificar. `/saldo/` sí consulta a Stripe en vivo, así que conviene llamarlo bajo demanda y no en el ciclo de polling.

7.6 Desarrollo local con Stripe CLI

```
stripe login
stripe listen --forward-to localhost:8000/api/payments/webhook/stripe/
# -> imprime whsec_...: cópialo a STRIPE_WEBHOOK_SECRET en el .env y reinicia el servidor
stripe trigger checkout.session.completed
# -> evento de prueba sin PagoStripe asociado: se registra y se responde 200 sin efecto
```

Para un recorrido completo, crea el checkout desde la app con las llaves `sk_test_` y paga con las tarjetas de prueba de Stripe (aprobada, rechazada, 3D Secure, fondos insuficientes). Nunca uses tarjetas reales en modo prueba.

7.7 Seguridad

Control	Implementación
Precio en el servidor	El monto sale de <code>Membresia.precio</code> ; lo que mande el frontend se ignora.
Autorización (IDOR)	El alumno solo paga y consulta lo suyo; el admin elige alumno; el maestro no tiene acceso.
Secretos	<code>STRIPE_SECRET_KEY</code> y <code>STRIPE_WEBHOOK_SECRET</code> solo en el <code>.env</code> ; nunca en Git ni en Angular.
Firma del webhook	<code>stripe.WebhookSignature.verify_header</code> sobre el cuerpo crudo, tolerancia 300 s.
Idempotencia	<code>EventoStripe.event_id</code> único + estatus del <code>PagoStripe</code> + <code>select_for_update</code> + <code>idempotency_key</code> en llamadas a Stripe.
Transacciones	El webhook procesa todo en <code>transaction.atomic</code> ; un fallo responde 500 y Stripe reintenta.
Integridad del importe	<code>amount_total</code> y <code>currency</code> del evento deben coincidir con lo guardado.
Rate limiting	<code>ScopedRateThrottle</code> 'checkout' (<code>THROTTLE_CHECKOUT</code> , 10/min por usuario).
Logs	<code>Logger payments</code> registra ids, tipo de evento y estatus; nunca llaves, tokens ni datos de tarjeta.
Datos de tarjeta	Ninguno: Stripe Checkout procesa la tarjeta; solo se guardan referencias <code>cs_ / pi_</code> .
Auditoría	El admin de Django muestra <code>PagoStripe</code> y <code>EventoStripe</code> en solo lectura.

8. Automatismos y tareas programadas

8.1 Señales

Disparador	Efecto
Alta de Alumno o Maestro	Crea su cuenta de acceso y su código QR (imagen PNG).
Alta de Alumno	Crea su Experiencia (ficha técnica) vacía.
Alta de Asistencia	Suma puntos con F(), recalcula la racha y evalúa insignias (con notificación).
Borrado de Asistencia	Devuelve los puntos y recalcula la racha.
Alta/borrado de Torneo	Recalcula el récord G-P-E de la Experiencia.
Alta de Pago	Envía el comprobante por correo; nunca revierte el pago si el correo falla.
Webhook de Stripe pagado	Crea el Pago (y con él, el comprobante) y ajusta la membresía del alumno.

8.2 Comandos de administración

Comando	Qué hace	Frecuencia sugerida
<code>python manage.py revisar_pagos --dias 5</code>	Refresca estatus de pagos y genera avisos de vencimiento y de vencido, sin duplicarlos.	Diario, 8:00
<code>python manage.py enviar_notificaciones_pendientes</code>	Envía los correos de avisos masivos pendientes.	Cada 5 min
<code>python manage.py seed</code>	Carga catálogos del cartel (horarios, disciplinas, planes, insignias) y datos demo.	Una vez

```
# crontab del servidor
0 8 * * * /ruta/venv/bin/python /ruta/manage.py revisar_pagos
*/5 * * * * /ruta/venv/bin/python /ruta/manage.py enviar_notificaciones_pendientes
```

8.3 Reglas de negocio destacadas

- **Estatus de pago calculado:** PAGADO o VENCIDO se derivan de fecha_vencimiento; solo PENDIENTE se respeta si se fija a mano.
- **Racha con tolerancia:** permite huecos de hasta 3 días entre sesiones (TOLERANCIA_RACHA_DIAS).
- **Doble check-in bloqueado:** una asistencia por clase y día, aunque falten disciplina u horario.
- **Pagos y ventas separados:** las ventas no mueven vencimientos ni el semáforo del alumno.
- **Imágenes con ruta relativa:** foto y QR salen como /media/..., para que funcionen detrás de proxy y desde el celular.

9. Pruebas y despliegue

9.1 Pruebas automatizadas

La suite tiene 222 pruebas (Django TestCase) que cubren modelos, reglas de negocio, permisos por rol, avisos, eventos, Evaluación MMA, ventas, media y la integración con Stripe (checkout, firma del webhook, idempotencia, activación, reembolsos y dashboard). Las pruebas de Stripe no llaman a la red: simulan la API y firman los webhooks con el mismo HMAC que usa Stripe.

```
python manage.py test          # suite completa
python manage.py test --parallel 4  # más rápido
python manage.py test payments  # solo pagos en línea
```

9.2 Lista de verificación para producción

Punto
[] DEBUG=False y SECRET_KEY real (activa HSTS, cookies seguras, redirección HTTPS).
[] ALLOWED_HOSTS, CORS_ALLOWED_ORIGINS y CSRF_TRUSTED_ORIGINS acotados a los dominios reales.
[] PostgreSQL (DB_ENGINE=postgresql) con respaldos programados.
[] pip install -r requirements-prod.txt; collectstatic; gunicorn config.wsgi:application.
[] Media (fotos y QR) en almacenamiento persistente (S3 o similar).
[] EMAIL_BACKEND SMTP real y cron de revisar_pagos y enviar_notificaciones_pendientes.
[] Llaves live de Stripe (sk_live_) en el .env del servidor, nunca en Git.
[] Endpoint de webhook HTTPS registrado en el Dashboard de Stripe con los eventos checkout.session.* y charge.refunded; su whsec_ en STRIPE_WEBHOOK_SECRET.
[] STRIPE_SUCCESS_URL y STRIPE_CANCEL_URL apuntando al dominio del frontend.
[] Prueba de extremo a extremo en modo test: checkout, webhook, activación, reembolso.
[] Check-in público protegido por red cerrada o token de dispositivo si la tablet sale de la red local.
[] python manage.py check --deploy sin advertencias.